3-DoF trajectory propagator

One fixed-step discrete model for either direction. R and V are three-element ECI vectors.

Case inputs `R_0, V_0, t_0, t_f, dt_nom`

Unit Delay

`x_k = [R_k; V_k; t_k]`

Initial condition

`[R_0; V_0; t_0]`

`Ts = dt_nom > 0`

`x_k`

Predictor

`Accel(R_k,V_k,t_k) → a1`

`R_p = R_k + Δt V_k`

`V_p = V_k + Δt a1`

Sum and Product blocks

`R_p,V_p,a1`

`x_k,Δt,t_p`

Corrector and update

`Accel(R_p,V_p,t_p) → a2`

`R_next = R_k + (Δt/2)(V_k + V_p)`

`V_next = V_k + (Δt/2)(a1 + a2)`

`t_next = t_p`

Stage 1 signals

Accel subsystem

AccelGrav(R)

Drag(R,V,t)

Σ → a

Evaluate at both stages.

`t_k`

`Δt, t_p`

Signed step

```
r  = t_f - t_k
Δt = sign(r) min(dt_nom, |r|)
t_p = t_f       if |r| ≤ dt_nom
      t_k + Δt  otherwise
```

`t_k = t_f`

Stop Simulation

Stop after this tick

Log accepted states

`R_k, V_k, t_k`

`acc_grav1, acc_drag1`

Also log drag diagnostics.

`x_next = [R_next; V_next; t_next]`

Build the discrete loop

Use fixed-step discrete, step size dt_nom and Stop Time inf. Pack the state with Vector Concatenate and unpack with Selector. Link both calls to one Accel subsystem. Log t_k as physical time.

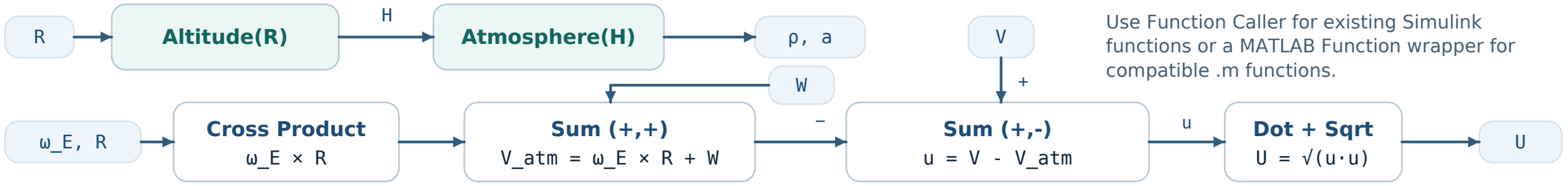
Check the update against your MATLAB code

The diagram uses Heun's explicit trapezoidal update. With signed accelerations, the sketch's a1 - a2 cancels constant gravity. Check the actual update before testing numerical parity.

Air-relative flow to drag acceleration

Inputs to each Drag instance are the stage position and velocity. Carry stage time or mode if the tables switch.

Atmosphere and relative velocity



Use Function Caller for existing Simulink functions or a MATLAB Function wrapper for compatible .m functions.

Use your existing Earth rate constant and W in ECI. Set W = 0 for co-rotating still air. For H-only profiles, use one 1-D lookup per property.

Flow quantities

Connect signals by name across rows.

Divide · Mach
`M = U / a`

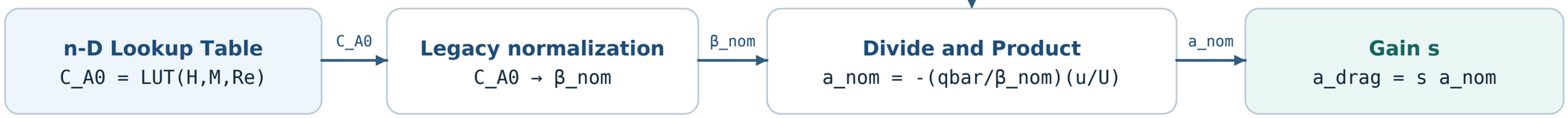
Product / Divide · Reynolds number
`Re = ρ U L_ref / μ`

Product / Gain · Dynamic pressure
`qbar = 0.5 ρ U²`

Axial lookup and force

H, M, Re as three inputs

Use qbar, u, U



For Re, supply dynamic viscosity μ or compute it from the temperature profile using the table's convention.

Zero airspeed

At U = 0, return zero drag. An If Action Subsystem can bypass the divide and lookup for that case.

Correction and diagnostics

Start with s = 1. Log C_A0, β_nom, s and β_eff = β_nom/s.

Wire the lookup, then compare truth cases

Match the legacy Mach-only baseline first. Then run the axial table and the drag gain through the same cases.

Legacy 1-D Lookup Table

```
beta = interp1(Mtable, Cdtable, Mach)
```

`Mtable`	Mach grid. Set as Breakpoints 1.
`Cdtable`	Sample values on that grid. Set as Table data.
`Mach`	Runtime input port. Connect U/a.

Use linear interpolation for this three-argument call.

A direct qbar/β term implies β in kg/m² with SI inputs. Trace one legacy lookup through the acceleration line to verify the stored quantity and any scaling.

Validated axial lookup at zero incidence

```
Example ports  u1 = H  |  u2 = M  |  u3 = Re
```

For a gridded table, match port order and breakpoints to the source array. If the axes are Mach, incidence and Re, feed α = 0 or use the zero-angle slice. Match interpolation and flag queries outside the grid.

Use the table's Reynolds convention and reference length. For unit Reynolds number, query ρU/μ in the grid's units.

For dimensionless C_A0 in the positive drag convention, use 1/β_nom = C_A0 × K_norm with your existing area-to-mass scaling. If the table supplies β directly, route it to β_nom.

If a case switches tables

Cap the physical step at the first switch time it would cross. Use the mode at the segment midpoint for both force evaluations. This works in either direction. Log the selected mode with β and drag.

Batch validation with one model run per truth case

Case k
`R_0, V_0, t_0, t_f, truth`

→

Simulink run
`log R,V,t and drag`

→

Match case timestamps
`score within overlap`

→

Error spread
`RMS, peak, endpoint`

Set case inputs with SimulationInput. Map the logs to sim.time (physical t) and sim.signals for the MATLAB comparator. Compare position, velocity, and available like-for-like β or coefficient and Re at truth timestamps. Score final position at a matched t_f and apply your margins per case.

To calibrate s, fit a shared positive gain to matched-state along-flow truth acceleration if available. With trajectory-only truth, fit normalized state errors. Evaluate the gain on held-out trajectories.

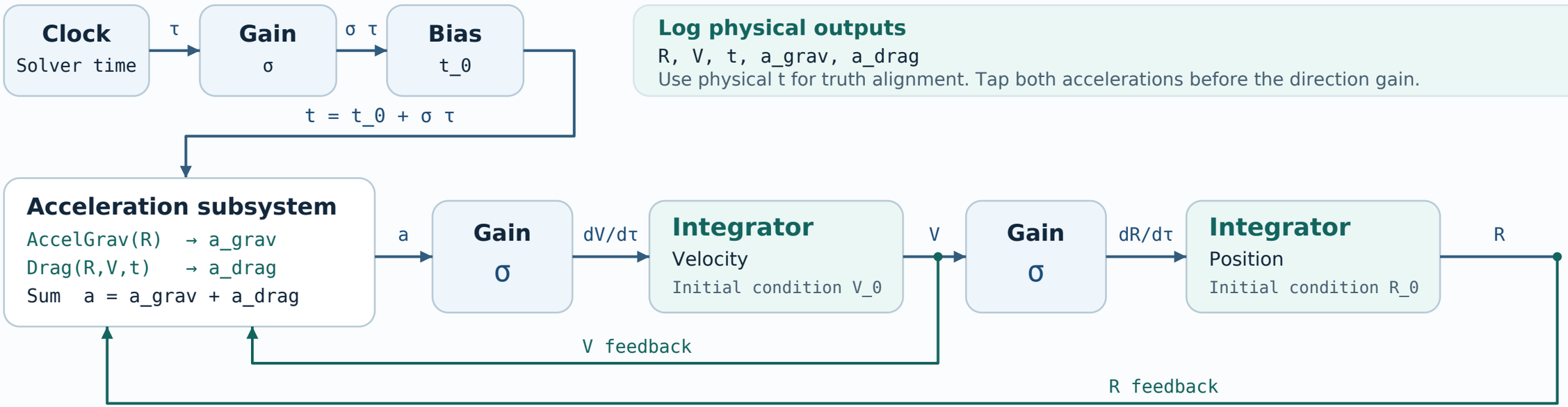
Continuous 3-DoF trajectory propagator

Two continuous Integrator blocks advance the three-element ECI position and velocity in either direction.

Case inputs `R_0, V_0, t_0, t_f`

`σ = sign(t_f - t_0)`

`T = |t_f - t_0|`



Log physical outputs

`R, V, t, a_grav, a_drag`

Use physical t for truth alignment. Tap both accelerations before the direction gain.

Model Settings → Solver

`Start time = 0` `Stop time = T`

Variable-step

`Solver ode45 (Dormand-Prince RK(4,5))`

Set Relative tolerance, Absolute tolerance and Max step size.
Start with Max step size = dt_nom to compare with sheet 1.

Fixed-step

`Solver ode4 (fourth-order Runge-Kutta)`

Set Fixed-step size (fundamental sample time) to a positive h.
Choose h so the duration T is an integer multiple of h.

Table switches

Use the active table throughout each integration segment.
At the switch, restart from final R, V and set t_0 to the switch time.
For ode4, use an integer number of steps in each segment.

Check convergence

Run the Heun baseline and continuous model with identical inputs.
Tighten ode45 tolerances and reduce Max step size, or halve h for ode4.
Compare the converged trajectories at the truth timestamps.

Map MATLAB functions to Simulink blocks

The examples use 3×1 ECI vectors. Match each wrapper call to your MATLAB function signature.

MATLAB function or operation	Simulink block	Ports and wiring
`AccelGrav(R)`	**MATLAB Function** Gravity path	`R (3×1) → a_grav (3×1)` Connect a_grav to the acceleration Sum block.
`Altitude(R)`	**MATLAB Function** Inside Drag	`R (3×1) → H (scalar)` Use H for Atmosphere and for any altitude-indexed lookup.
`Atmosphere(H)`	**MATLAB Function** Or one 1-D Lookup Table per H-only property	`H (scalar) → ρ, a (scalars)` Use density ρ for qbar and Re. Use sound speed a for Mach.
`Drag(R,V,t)`	**Subsystem** Three Inports and one Outport	`R (3×1), V (3×1), t (scalar) → a_drag (3×1)` Wire physical t from sheet 4. Build the drag path on sheet 2.
`interp1(Mtable,Cdtable,Mach)`	**1-D Lookup Table** Legacy Mach-only model	`Mach (scalar) → β (scalar)` Set Breakpoints 1 to Mtable and Table data to Cdtable.
Validated axial table	**n-D Lookup Table** Axial model on sheets 2 and 3	`H, Mach, Re → C_A0 (example port order)` Match the source axes and feed C_A0 to legacy normalization.

`propagate_traj(R_0,V_0,t_0,t_f,dt)` **→ Top-level model on sheet 4** `Log R, V, t, a_grav, a_drag`

R_0 and V_0 set Integrator initial conditions. t_0 and t_f define the physical clock and Stop time.
Use abs(dt) as a starting Max step size for ode45 or Fixed-step size for ode4. See sheet 4 for endpoint alignment.

Wrap a callable .m function

```
function a_grav = fcn(R)
    a_grav = AccelGrav(R);
end
```

In a MATLAB Function block, the signature defines the ports.
Use Size [3 1] for R and a_grav in this example.
Direct calls use code-generation-compatible .m files on the MATLAB path.
If gravity is local to propagate_traj.m, put it in its own AccelGrav.m file.

Call a named Simulink Function

Add a Function Caller and set its Function prototype.

`a_grav = AccelGrav(R)`

Match the function name, argument names and port sizes.

Connect the continuous force path

For the wrappers on sheet 4, set Update method to Continuous.
Feed a_grav and a_drag into the Sum block with signs ++.
Connect the sum to the direction gain before the velocity Integrator.